

Avoid Using JOIN in MySQL

Kousuke Hidaka, 2024/10/16

Background

While developing admin-v2, I received various advice from Mr. Furuno, one of which was, "MySQL is not a database that excels at JOIN operations, so try to avoid using JOIN as much as possible."

I had never heard this before, so I decided to investigate further.

Why MySQL is not suitable for JOIN

Conclusion

Because it only supports the Nested Loop join algorithm.

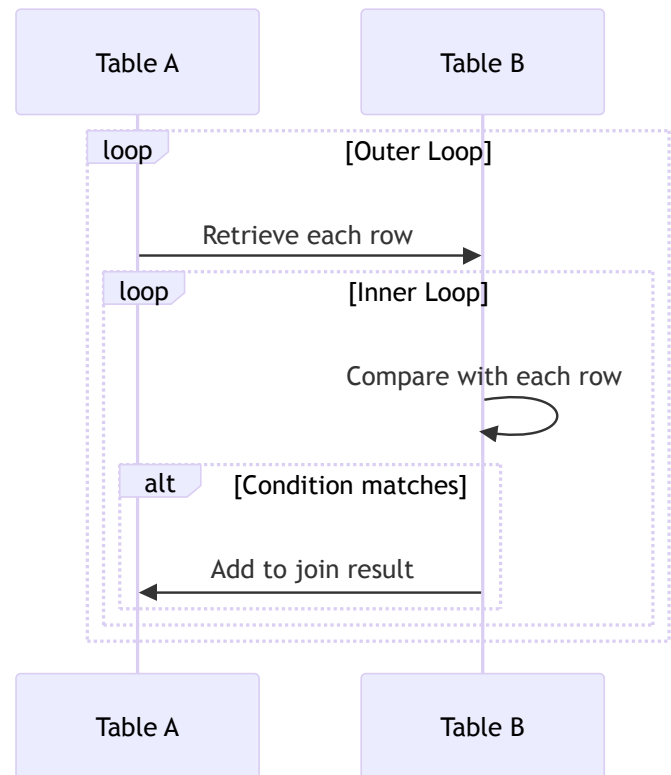
Types of Join Algorithms

- Nested Loop
- Hash Join
- Sort Merge Join

Nested Loop

1. Retrieve columns from the source table
2. Search the target table using the columns obtained in step 1
3. Join the columns obtained in step 2 to the target table

In other words, the computational complexity is $O(N*M)$



This algorithm is effective, especially for small tables or when indexes are properly set, but performance may degrade with large datasets.

Compared to the other two algorithms, this algorithm can use any comparison operator.

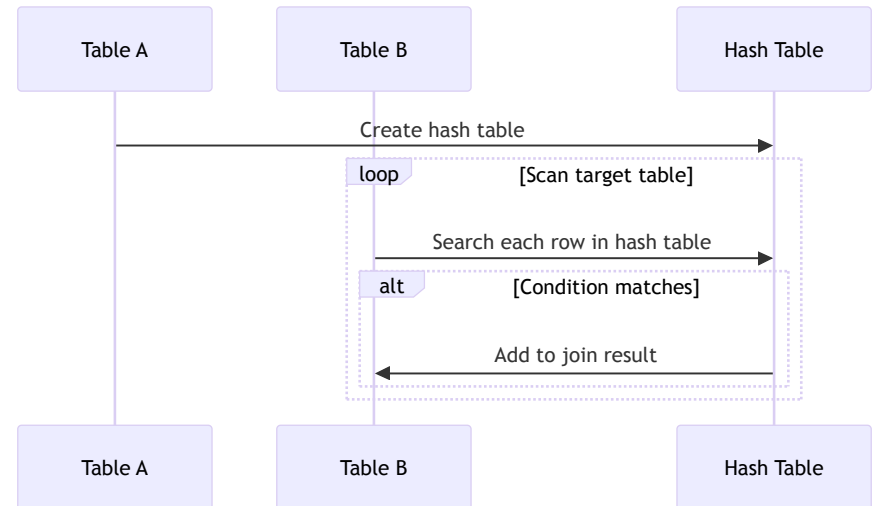
Hash Join

1. Retrieve columns from the source table and create a hash table
2. Retrieve columns from the target table and search for matching rows using the hash table
3. Add matching rows to the join result

The computational complexity of this algorithm is $O(N+M)$, making it efficient for large datasets.

Hash Join is effective, especially for large datasets or when there are no indexes, and operates more efficiently than Nested Loop.

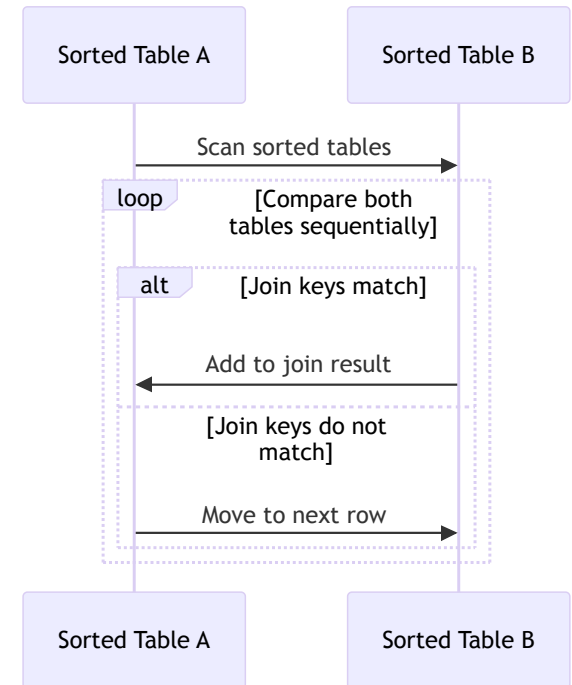
However, because it compares hash values, it only supports the "=" comparison operator. Also, since Hash Join creates a hash table for comparison, it uses a lot of memory, so care must be taken to avoid memory shortages.



Sort Merge Join

1. Sort both the source and target tables by the join key
2. Sequentially scan the sorted tables and join rows with matching join keys
3. Add matching rows to the join result

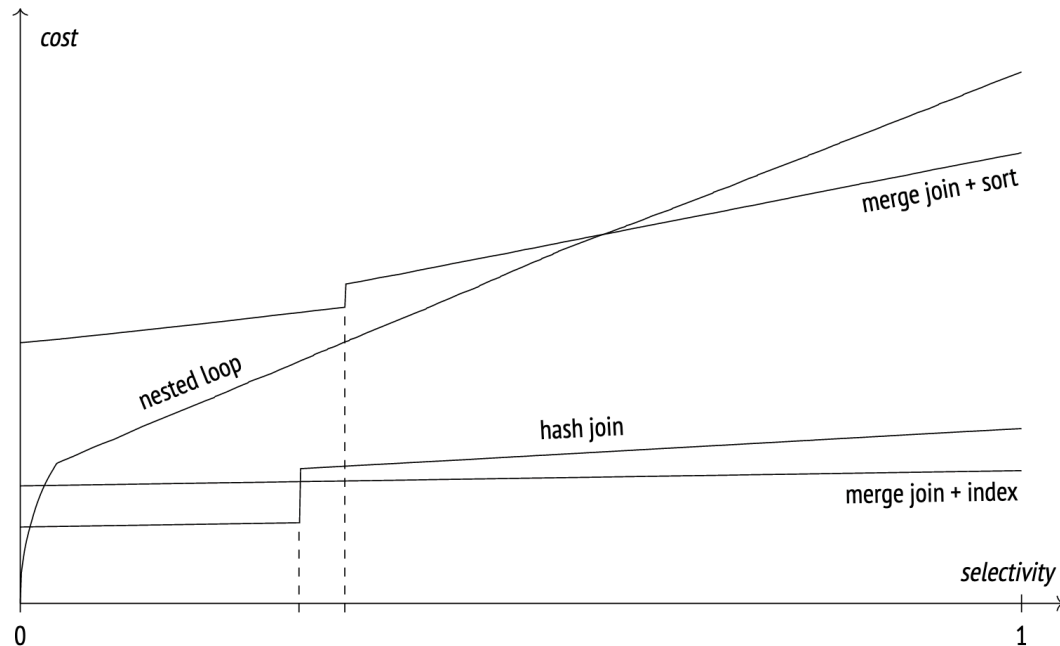
The computational complexity of this algorithm is $O(N \log N + M \log M)$. Although sorting is required, it is very efficient when the join keys are already sorted.



Sort Merge Join is effective, especially when the join keys are already sorted or for joins involving range conditions. Although it has a high initial cost due to sorting, it operates very efficiently on sorted data compared to Nested Loop and Hash Join.

Comparison of Data Volume and Cost for Each Algorithm

- For "=" joins, hash join is generally the fastest, and for large volumes, merge join + sort is the fastest.
- As the volume increases, the performance of nested loop deteriorates, so it is not preferred except for joining small datasets or when using operators other than '='.



Note: The steps in "hash join" and "merge join + sort" occur when data spills over to the disk due to insufficient memory.

Comparison of Supported Join Algorithms in Major Databases

Database	Nested Loop	Hash Join	Sort Merge Join
MySQL	○	○ (from 8.0.18)	×
PostgreSQL	○	○	○
Oracle	○	○	○

How to Handle Large Table Joins in MySQL for admin-v2

Prerequisites

- Do not want to modify the database
 - Because it is referenced by various other products
 - A large-scale database replacement project is planned

Approach

- Put the load on the application side
 - Extract records from each table individually and merge them to achieve the join

Supplement

- In cloud-based systems, considering the ease of scaling in/out, it is easier to put the load on the app rather than the DB, making it easier from an infrastructure perspective.

Summary

- There are three types of RDB table join algorithms
 - Nested Loop
 - Hash Join
 - Sort Merge Join
- MySQL only supports Nested Loop, so it may not be suitable for handling large amounts of data
 - However, since version 8.0.18, Hash join is available, so MySQL is not necessarily less user-friendly in terms of join algorithms

```
SET optimizer_switch = 'hash_join=on';
```

References

- [Differences and Performance Comparison of Nested Loop/Hash/Sort Merge Joins](#)
- [Queries in PostgreSQL: 7. Sort and merge](#)
- [\[SQL\] About the Three Types of Join Algorithms](#)